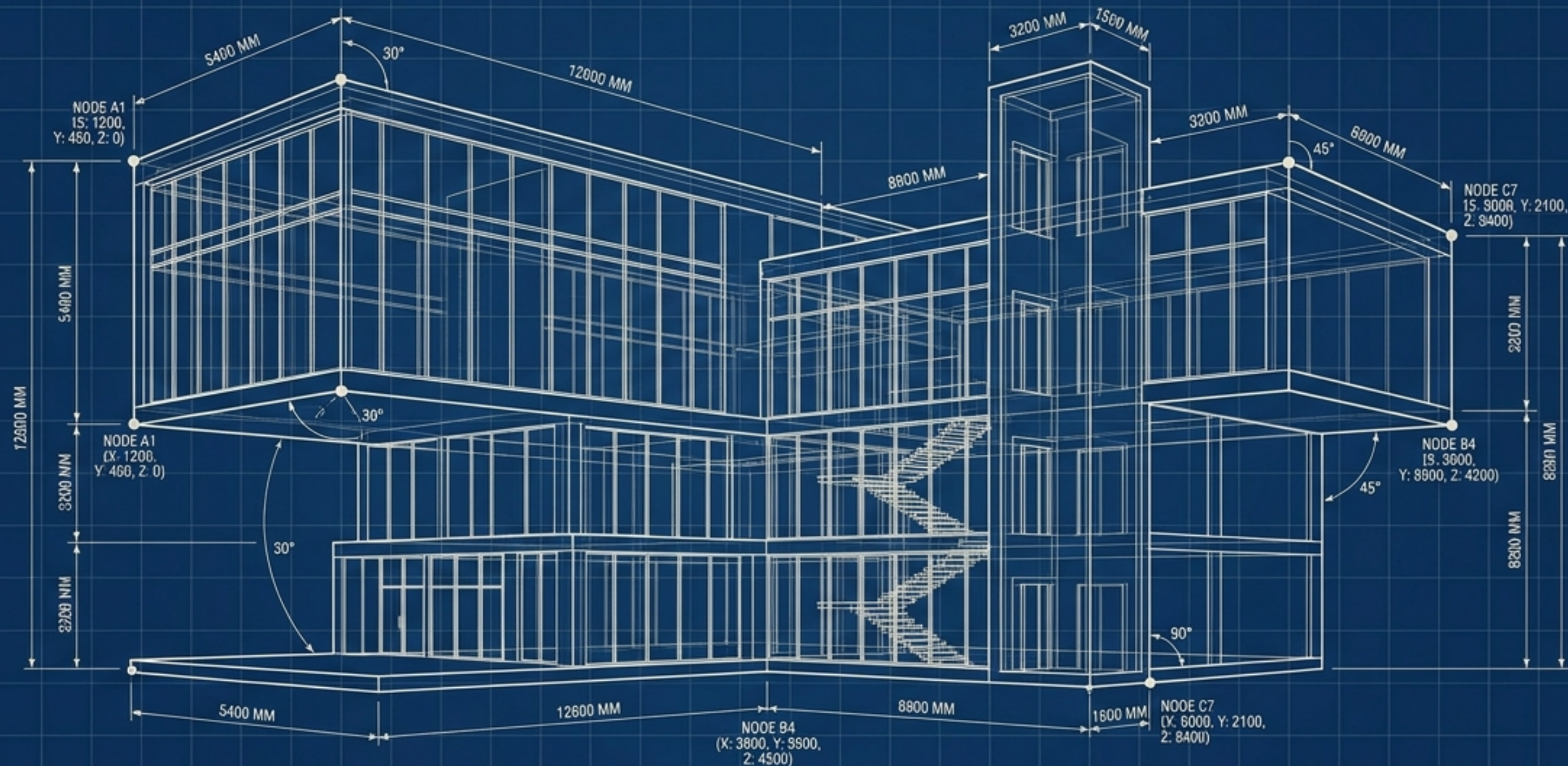
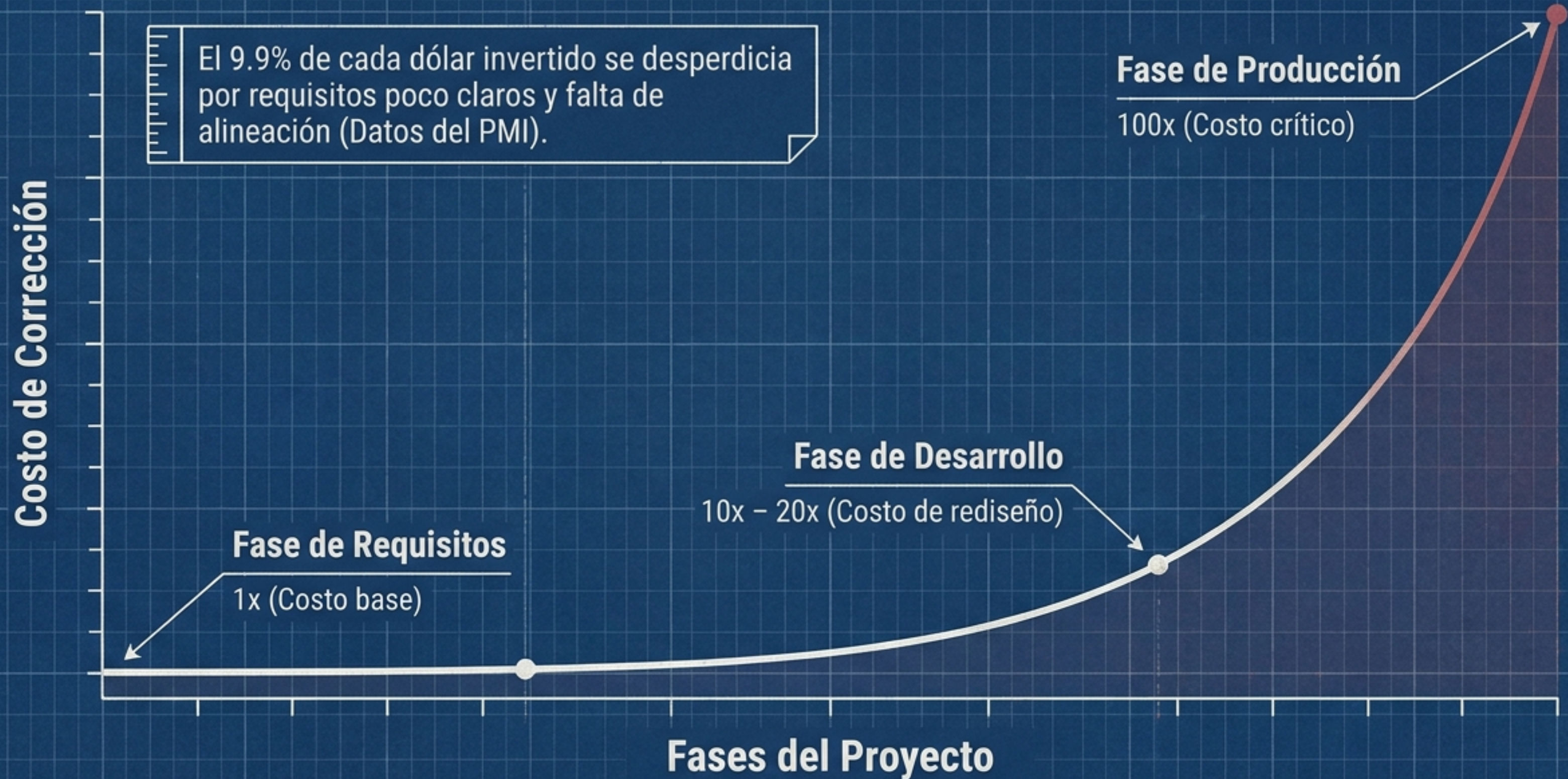


EL PLANO DEL ÉXITO EN SOFTWARE

SRS: Sencillo, directo y al punto.



El costo real de la ambigüedad en los requisitos



Por qué fracasan los proyectos de software



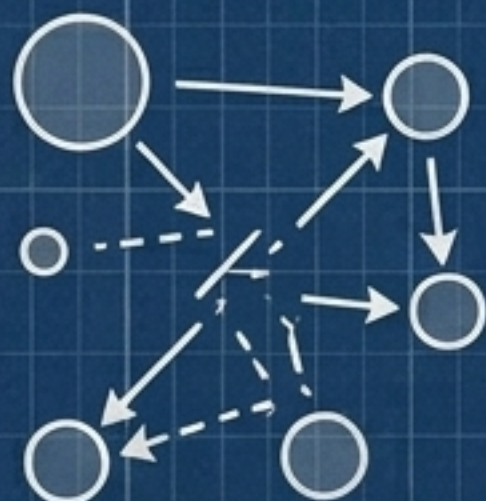
Desvío de Alcance (Scope Creep)

Funciones no planificadas y cambios continuos que destruyen el presupuesto inicial.



Retrasos en el Desarrollo

Equipos perdiendo tiempo aclarando expectativas vagas y rehaciendo código.



Fricción y Desalineación

Desarrolladores, clientes y control de calidad (QA) asumiendo objetivos diferentes.



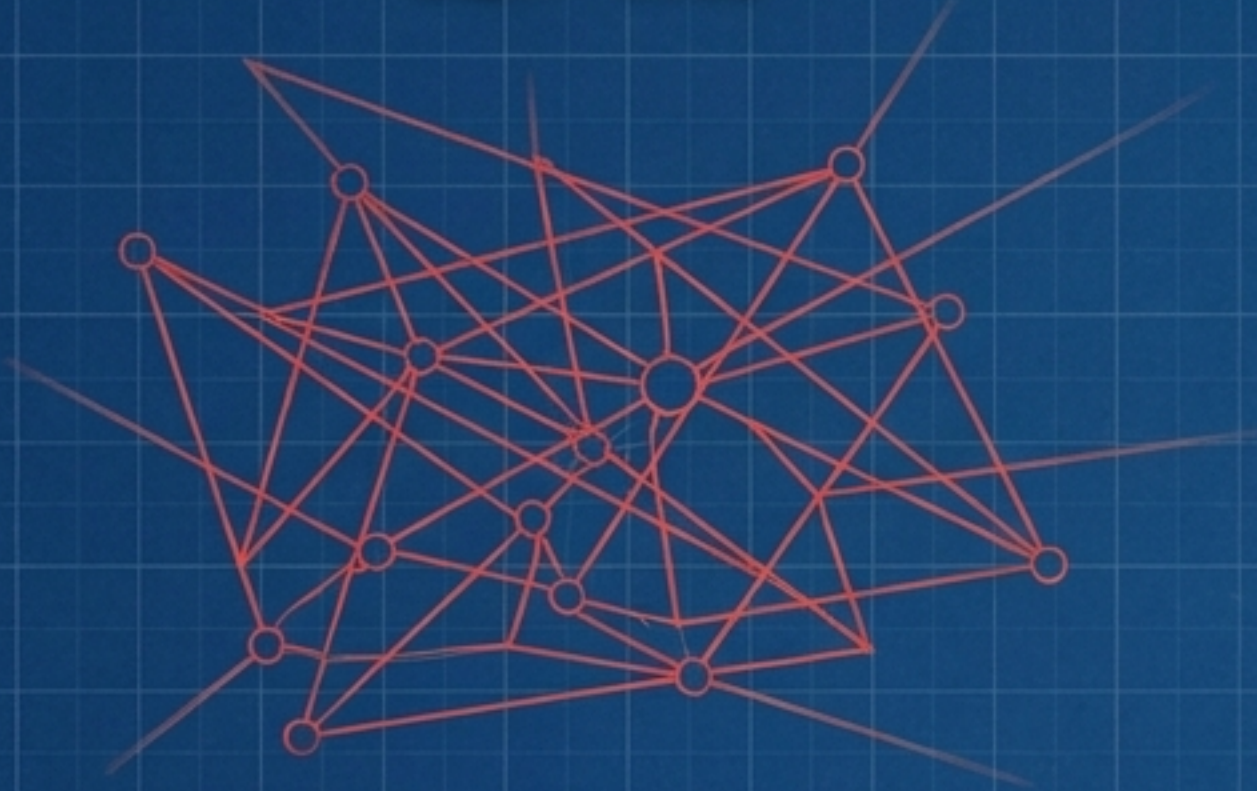
Riesgos de Seguridad y Cumplimiento

Multas por GDPR o brechas de datos al ignorar protocolos desde el día uno.

Ejemplo real: El Proyecto Libra (Reino Unido) pasó de un presupuesto de £146M a £390M por un análisis de requisitos inadecuado.

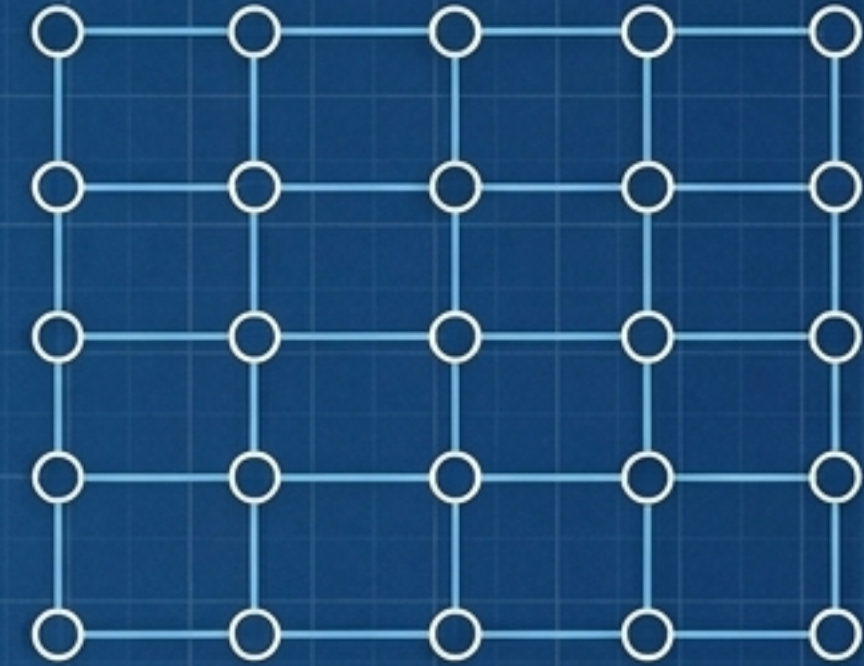
Del caos a la certeza estructural

Sin SRS



- Presupuesto ciego
- Expectativas vagas
- Desarrollo reactivo
- Pruebas basadas en opiniones

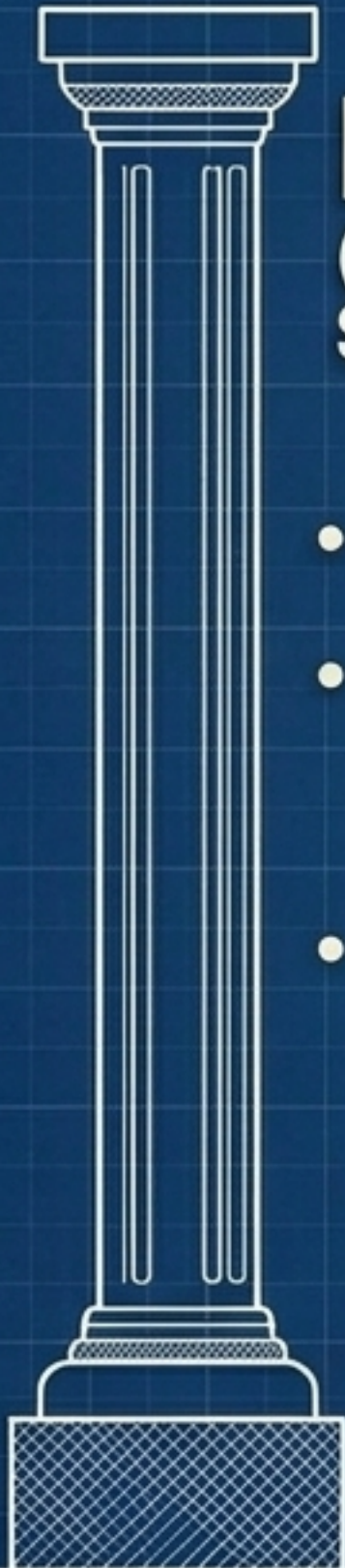
Con SRS



- Costos predecibles
- Límites claros (Scope cerrado)
- Ejecución estructurada
- Pruebas basadas en métricas

Un SRS (Software Requirements Specification) es el plano arquitectónico de su software. Define exactamente QUÉ hará el sistema sin dictar CÓMO programarlo.

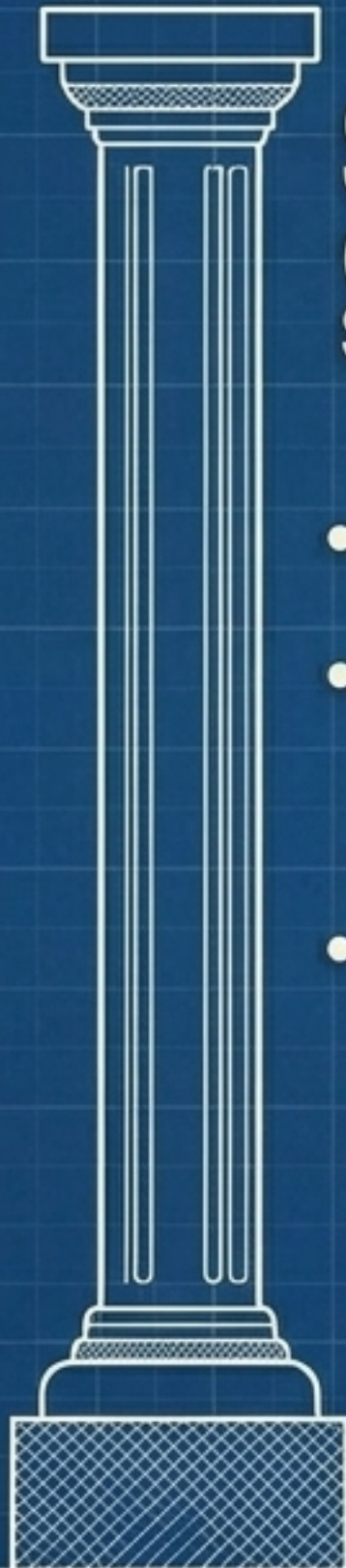
La jerarquía de los requisitos



BRS

(Business Requirements Specification)

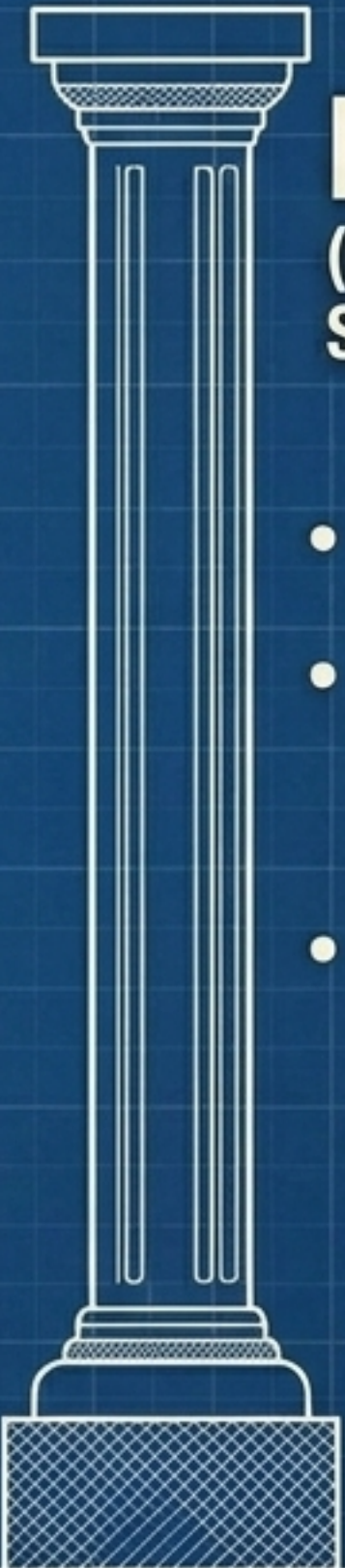
- **Enfoque:** El Por Qué
- **Audiencia:** Líderes de negocio, inversores, clientes.
- **Objetivo:** Define la visión, los objetivos comerciales y el retorno de inversión (ROI).



SRS

(Software Requirements Specification)

- **Enfoque:** El Qué
- **Audiencia:** Arquitectos de sistemas, líderes técnicos, PMs.
- **Objetivo:** Traduce la visión de negocio en lógica de sistema, límites y capacidades técnicas.



FRS

(Functional Requirements Specification)

- **Enfoque:** El Cómo
- **Audiencia:** Desarrolladores, diseñadores UI/UX, equipo QA.
- **Objetivo:** Detalla flujos de trabajo, interacciones del usuario y módulos de código específicos.

Anatomía de un SRS de alto nivel

1. Introducción y Alcance

El contexto. Propósito del sistema, audiencia objetivo, y los límites estrictos de lo que se construirá (y lo que no se construirá).

2. Requisitos Funcionales

El comportamiento. Entradas, salidas, reglas de negocio e interacciones del usuario.

3. Requisitos No Funcionales (Atributos de Calidad)

El rendimiento. Velocidad, seguridad, escalabilidad, y fiabilidad del sistema bajo estrés.

4. Restricciones y Dependencias

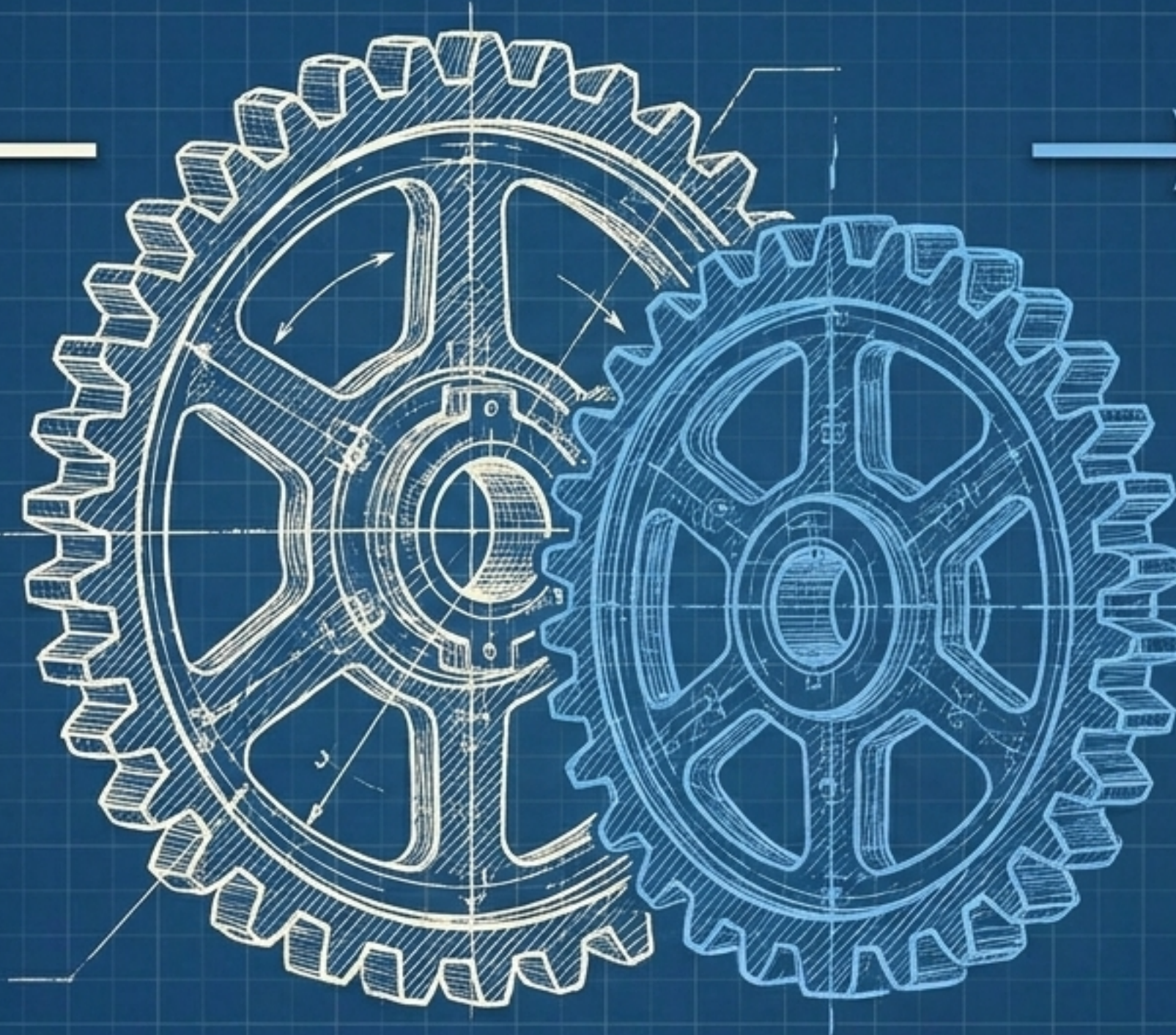
Las bases. Limitaciones tecnológicas, integraciones de hardware/software, y cumplimiento normativo (ej. GDPR, ISO 27001).

El equilibrio vital: Funcional vs. No Funcional

Requisitos Funcionales

(Lo que el sistema hace)

- Comportamiento observable.
- Ejemplo: "El sistema debe permitir al usuario iniciar sesión con nombre de usuario y contraseña."
- Riesgo si falla: Falta una característica clave que el cliente pidió.



Requisitos No Funcionales

(Cómo rinde el sistema)

- Atributos de calidad y restricciones.
- Ejemplo: "El sistema debe garantizar un 99.9% de tiempo de actividad (uptime) anual."
- Riesgo si falla: El sistema funciona, pero colapsa bajo carga o es vulnerable a hackeos.

La anatomía de un requisito perfecto

Cero ambigüedad. Cero suposiciones.



El error de la vaguedad

El sistema debe ser rápido y fácil de usar.

El problema: ¿Qué significa "rápido"? Un desarrollador asume 5 segundos; el cliente asume 0.5 segundos. Es imposible de probar.

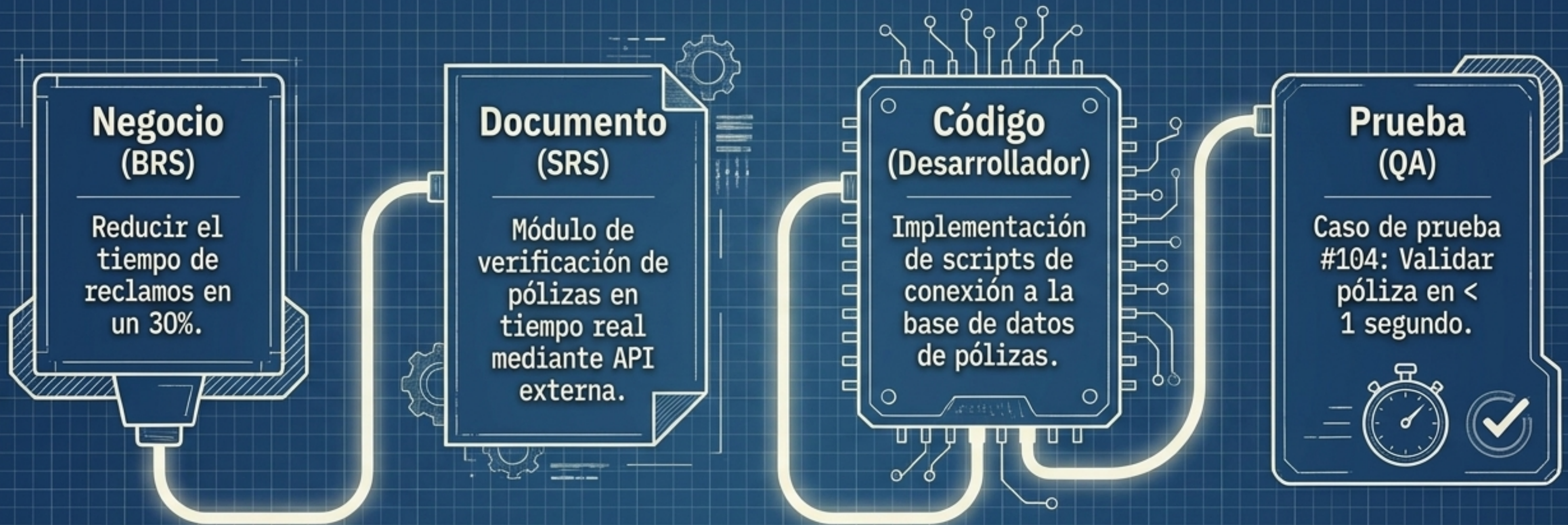


Precisión de ingeniería

El sistema debe procesar 1,000 transacciones por segundo bajo carga máxima con un tiempo de respuesta menor a 2 segundos.

El valor: Es CLARO (sin adjetivos vagos), COMPLETO (define el contexto de carga máxima), y VERIFICABLE (QA puede medir exactamente 2 segundos).

Trazabilidad: El hilo conductor del proyecto



La matriz de trazabilidad asegura que CADA LÍNEA DE CÓDIGO justifique un objetivo de negocio. Nada se construye por accidente.

Una única fuente de la verdad para todo el equipo



Desarrolladores

Construyen con certeza.

Reciben un mapa arquitectónico claro que reduce el retrabajo.



Project Managers (PMs)

Gestionan sin sorpresas.

Utilizan el documento como línea base para controlar el alcance, el tiempo y el presupuesto.



Control de Calidad (QA)

Prueban con precisión.

Utilizan el SRS para generar casos de prueba exhaustivos basados en métricas exactas.



Clientes

Reciben lo que pidieron.

Firman un contrato técnico que garantiza que su visión de negocio se ejecutará fielmente.

Reglas de oro para un SRS efectivo



Lenguaje Estandarizado

Cero adjetivos técnicos vagos. Evite palabras como 'aproximadamente', 'eficiente' o 'flexible'. Use medidas cuantitativas.



Control de Versiones

El SRS es un documento vivo (Docs as Code). Utilice herramientas como Jira, Confluence o Git para mantener un historial. Cada actualización deja un rastro.

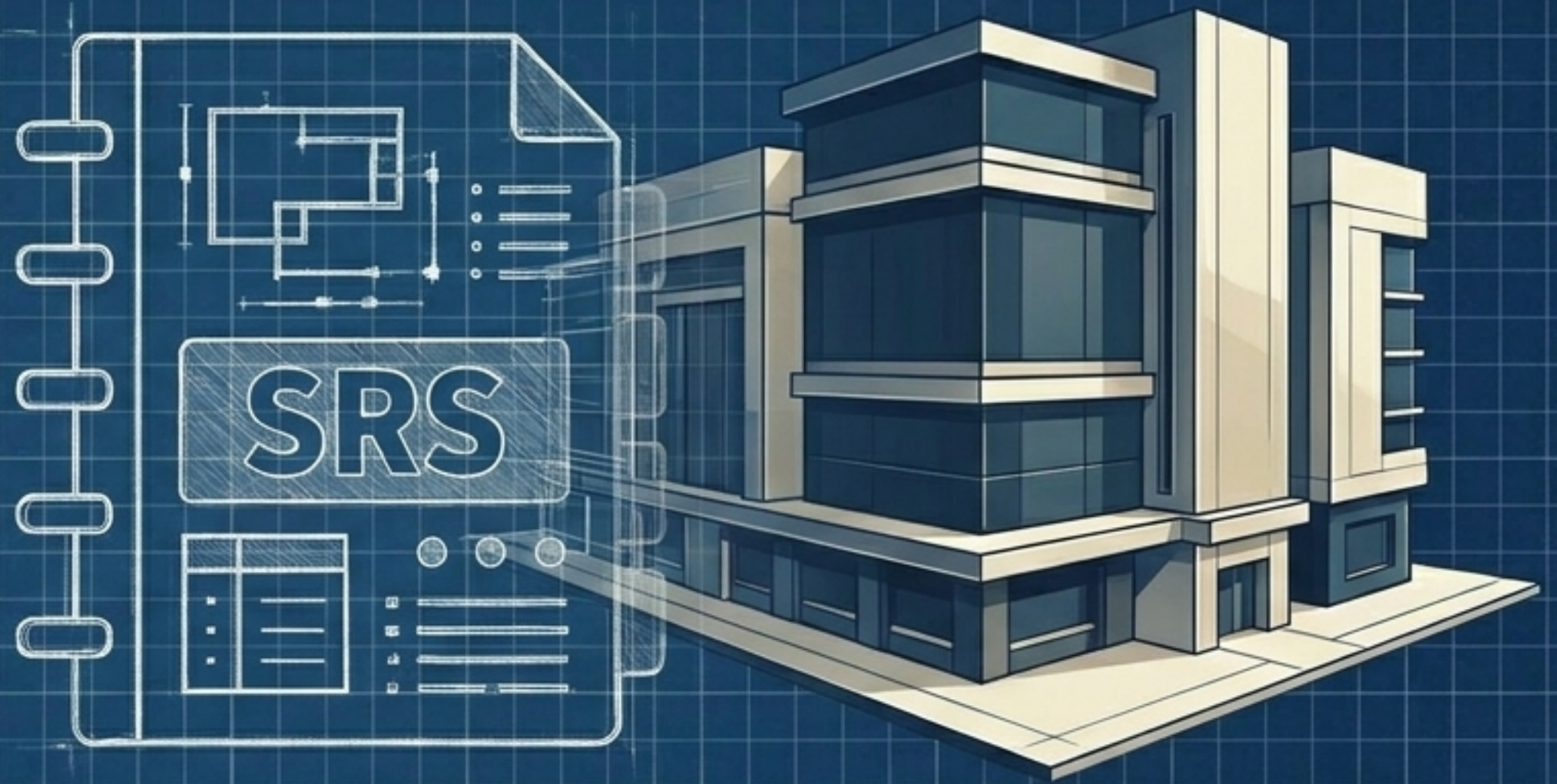


Iteración y Validación

No espere a que esté 100% terminado para validarlo. Involucre a desarrolladores y testers temprano (Agile) para asegurar la factibilidad técnica.

El SRS no es burocracia. Es protección.

- Elimina la ambigüedad y el “yo creía que...”
- Frena la expansión descontrolada del alcance (Scope Creep).
- Garantiza que el software cumpla con normativas de seguridad y rendimiento.



Invertir en los requisitos hoy es el único camino para asegurar el éxito, el presupuesto y los plazos de mañana. Cero excusas, cero ambigüedad.